

LM - Lab 4

Donald Gera (257565)

University of Trento

donald.gera@studenti.unitn.it

1. Introduction

In **Part A**, I trained a GPT-2-like model from scratch and evaluated incremental changes to its configuration. In **Part B**, I started from pretrained GPT-2 weights and manually implemented Low-Rank Adaptation (LoRA) [2]. In both parts, models were trained on the Penn Treebank dataset and configurations were compared using perplexity (PPL) as the main evaluation metric, and the selected best models were evaluated on the test set.

2. Implementation details

Part A. The experiments were conducted incrementally. Starting from the baseline architecture, I selected the learning rate while leaving the remaining hyperparameters unchanged. I then performed a greedy search over `d_model`, `n_heads`, `num_layers`, and `ff_dim`: each parameter was tuned separately, while the best values identified in the previous steps were preserved. Since the resulting model was larger than the baseline, I carried out a second learning-rate search. I subsequently introduced **dropout** at four points of the architecture: after the sum of token and positional embeddings, after the computation of the attention weights, after the attention output projection, and after the last linear layer of the feed-forward block. As a final modification, I introduced **weight tying** by reusing the token embedding matrix in the output layer. Consequently, the same learned token representations are used both to encode the input tokens and to compute the scores for the next-token prediction.

Part B. I extended pretrained GPT-2 [3], by implementing LoRA on top of it. For an original frozen matrix W , LoRA represents its trainable update as

$$\Delta W = \frac{\alpha}{r} BA$$

For hidden size d , the manually added matrices have shapes $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{d \times r}$. The rank r controls the capacity and parameter count of the adapter, while α controls the magnitude of its contribution. The factor α/r normalizes the update in different ranks. I added independent adapters to the query, key, and value projections. During the forward pass, the scaled term $(\alpha/r)B(Ah)$ is added to each corresponding frozen projection output. The A matrices are normally initialized, and the B matrices are zero-initialized, as per implementation in the corresponding paper. All pretrained parameters are frozen, and only the LoRA adapter parameters are passed to the optimizer.

3. Results

Part A. Table 1 reports the incremental experiments. The architectural search improved test PPL from 45.11 to 37.65. Re-adjusting the learning rate further reduced it to 36.01. Dropout ($p = 0.10$ is the best dropout value) and weight tying were both

tested but discarded because they slightly worsened development and test PPL.

Table 1: **Part A** - incremental experiments. The configuration column reports `d_model` / `n_heads` / `num_layers` / `ff_dim`.

Experiment	LR	Configuration	Dev PPL	Test PPL
Baseline	$5 \cdot 10^{-4}$	20 / 1 / 1 / 20	51.57	45.11
Greedy Tuning	$5 \cdot 10^{-4}$	40 / 4 / 4 / 160	42.52	37.65
Final Baseline	$2 \cdot 10^{-3}$	40 / 4 / 4 / 160	40.27	36.01
Dropout ($p = 0.1$)	$2 \cdot 10^{-3}$	40 / 4 / 4 / 160	40.34	36.11
Weight tying	$2 \cdot 10^{-3}$	40 / 4 / 4 / 160	40.92	36.48

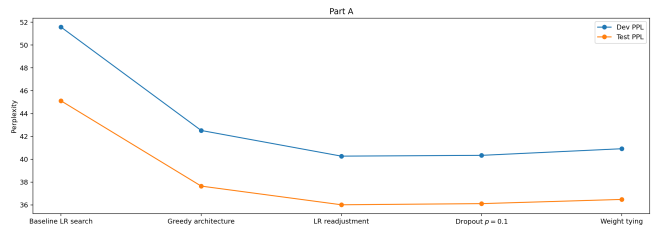


Figure 1: **Part A** - PPL across the incremental experiments.

Part B. Figure 2 reports the LoRA sweep. Increasing the rank consistently improved PPL within the tested grid. As expected, increasing the number of trainable parameters generally improves performance, since adapters with a larger rank can represent a more expressive update to the frozen pretrained weights. This must be balanced against the additional capacity and computational cost. Increasing α does not add trainable parameters, but changes the relative strength of the learned update; its effect therefore needs to be tuned jointly with the rank.

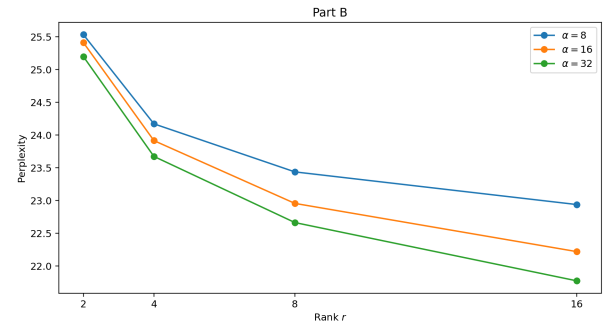


Figure 2: **Part B** - PPL for the LoRA sweep.

The best **Part B** model uses $r = 16$ and $\alpha = 32$. It trains 884,736 adapter parameters, corresponding to 0.706% of the

model, and reduces test PPL from 36.01 in **Part A** to 19.78: a 45.08% relative reduction.

4. References

- [1] M. Rizzoli, “Natural language understanding labs,” <https://github.com/massimo-rizzoli/NLU-2026-Labs>, 2026, university of Trento, accessed: 2026-06-02.
- [2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” *arXiv preprint arXiv:2106.09685*, 2021.
- [3] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” OpenAI, Tech. Rep., 2019.